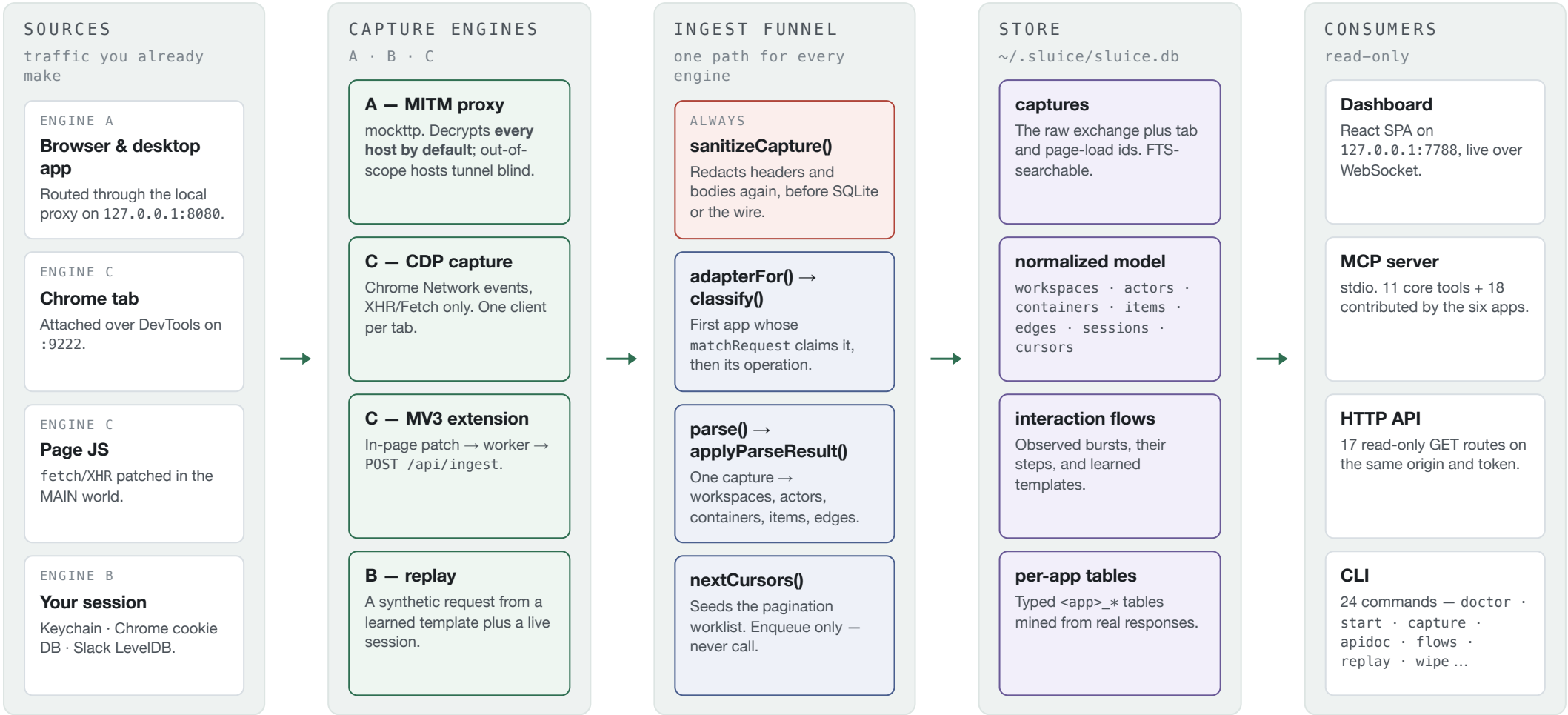
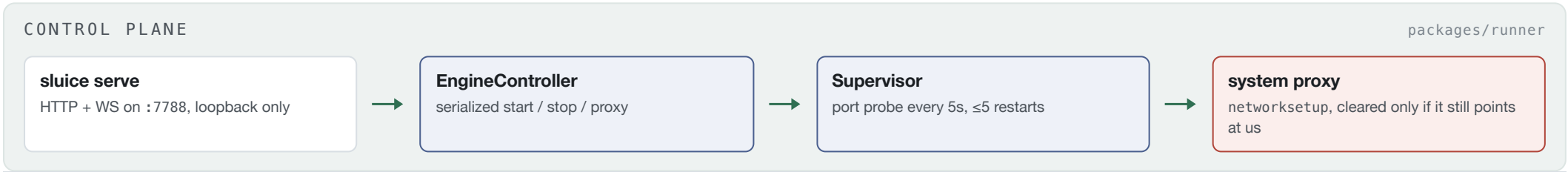


The whole pipeline

Where traffic comes from, what records it, and what reads it back



Interception is scoped, not filtered: a host outside the scope is CONNECT-tunnelled as raw bytes, so no row is ever written for it. Because the default is to decrypt everything, a service with no adapter still lands in captures – unattributed, redacted and visible – rather than silently missing.



Capture engines

How each engine decides what it is allowed to see

ENGINE A — MITM PROXY

interceptor/src/mitm-engine.ts

a local HTTPS proxy you route a client through

Routed client

Browser via the macOS system proxy, or a desktop app launched with `--proxy-server=127.0.0.1:8080`.



TLS scope decision

Default: decrypt every host. Any `--host / interceptHosts` entry — or `interceptAllHosts: false` — narrows it to the adapters' hosts plus those, each with a `*.host` wildcard.



In scope → decrypted

Passed straight through, never rewritten. Text bodies only, capped at 5 MB. WebSocket text frames become their own captures.

Out of scope → tunnelled

CONNECT'd as raw bytes. Sluice cannot read it, so it cannot store it — a stronger guarantee than redacting it afterwards.

ENGINE C — CDP

cdp-engine.ts

sluice capture

Attach, don't proxy

Chrome runs with `--remote-debugging-port=9222` and its own profile. The engine issues no requests of its own — no CA, no proxy.

XHR / Fetch only

One CDP client per page target, re-scanned every 2s. Documents, scripts, images, media and beacons are dropped before a row exists.

ENGINE C — EXTENSION

packages/extension

serve --ingest

Two worlds, one bridge

MAIN-world patch of `fetch/XHR` → `postMessage` → ISOLATED-world bridge that re-validates and re-clips at 512 KiB.

Default-deny hosts

The MV3 worker sends nothing until an endpoint, an ingest token and an explicit host list are set. Batches of ≤ 500 , flushed at 50 or 2s.

ENGINE B — YOUR SESSION

app-*/src

read on demand, never stored

Slack

The `xoxc` token from Slack desktop's LevelDB plus the `d` cookie, decrypted with the *Slack Safe Storage* Keychain item.

Trello · Loom · LinkedIn

Chrome's per-profile cookie DB via *Chrome Safe Storage*; LinkedIn also derives a `csrf`-token from `JSESSIONID`. Gmail and `fast.com` need none.

WHAT ENGINE A KEEPS SEPARATE

each step is its own deliberate command

Generating the CA

`sluice start` mints `sluice-ca.key/.cert` under Application Support if they are missing. That alone decrypts nothing.

Trusting the CA

`sluice ca-install` adds it to the login keychain — a separate, deliberate step, reversed by `ca-uninstall`.

Isolating the engine

`serve --isolated` runs the proxy in its own process, speaking NDJSON over fd 3. It carries no adapters: attribution and parsing stay in the parent.

Every engine redacts at its own boundary — `redactUrl`, `redactHeaders`, `redactText` — before the capture leaves the module, and the runner redacts again on `ingest`. Attribution is separate: an unrecognised service is stored with `adapter_id` NULL rather than dropped.

Ingest & normalization

One funnel, one redactor, one shape for every service

THE FUNNEL

runner/src/server.ts · ingestCapture()

live capture, extension POST, replay and sync all enter here

1 · sanitize

Redact headers and both bodies **again**, before SQLite or the wire.

2 · attribute

The capture's own adapterId, else the first app whose matchRequest claims it.

3 · classify

The adapter's semantic operation, or operationName(path) as the fallback.

4 · store & parse

Insert the row, then parse() → applyParseResult(). Neither may throw.

5 · seed & broadcast

Enqueue nextCursors() seeds, push capture.new, debounce the catalog and materialize.

Redaction runs **before** attribution — which is why each app's token shapes are folded into one global policy at import time rather than applied per adapter. Pause stops live ingest only; replay and sync still write.

REDACTION POLICY

core/src/redact.ts

Generic rules

Masks authorization · cookie · set-cookie · proxy-authorization, any credential-shaped header name, Bearer ... values, and token=-style fields.

App-registered shapes

Slack adds xox*- and two headers; Gmail adds its XSRF header. Registered for *all* traffic, not just that app's.

publicParams

The one rule that **preserves** a value: fast.com's token is public, and masking it would make the capture unreplayable.

NORMALIZED MODEL

five types every app maps onto

Workspace → Container → Item

A workspace holds containers (channel · dm · group · board · project · thread · other) which hold items (message · page · issue · other).

Actor · Edge

People, and the relationships between any two entities — authored, member-of, in-label, mentions.

Provenance

Items carry the capture ids they came from, so every row can be traced back to the bytes that produced it.

THE STORE

core/src/schema.ts

~/.sluice/sluice.db
· WAL

11 tables + 2 FTS

captures · workspaces · actors · containers · items · edges · sessions · cursors · interaction_flows(+steps) · flow_templates

Contentless FTS

captures_fts and items_fts index the plaintext but store no second copy; results join back on rowid.

Nowhere to put a secret

There is no credentials table. Only a redacted session descriptor — kinds, never values — is persisted.

CARTOGRAPHER

packages/cartographer

what the store makes of it

API catalog

sluice apidoc groups captures by method + host + path into a Markdown endpoint reference. Scope it with --host.

Per-app tables

materialize() mines **top-level response arrays** — not parsed entities — into typed <app>_* tables, rebuilt as traffic arrives.

Flows & templates

learn-flows clusters bursts into flows and distils multi-step templates with per-step timing and parameter bindings.

Schema changes are additive by construction: CREATE TABLE IF NOT EXISTS does nothing to an existing database, so a new column goes in both the create statement and the additive-column map, and its index runs only after the ALTERs.

Replay

Reading your session without being able to act as you

CALLERS

the rails sit below all of them, so a modified frontend cannot route around them

CLI

`sluice replay --flow --all sync`

Dashboard

`replay.run` over the WebSocket

MCP tools

`replay · sluice_replay_flow`

App tools

`ctx.replay` — never a bare fetch

REPLAY SAFETY RAILS

`interceptor/src/replay-policy.ts · core/src/replay-deny.ts`

three independent limits; a request must clear all three

1 · Method

GET, HEAD and POST only. PUT / PATCH / DELETE are refused outright — they can only mutate.
→ `method_not_allowed`

2 · Operation

Matched against the path, query **and** body, because a service can POST for an ordinary read — `chat.post*` · `admin.*` · `files.upload|delete` · `conversations.create|invite` · `reactions` · `oauth|auth.token` · GraphQL mutation.
→ `operation_not_allowed`

3 · Budget

A process-global token bucket — 60 requests per 60s — plus single-flight serialization, shared by the CLI, the dashboard, sync and every MCP tool.
→ `rate_budget_exhausted`

Sluice can read your session; it cannot act as you. That property is enforced here rather than in the UI, and the same three gates apply to every step of a multi-step flow.

BUILD AND SEND

`cartographer/src/faithful.ts · interceptor/src/replay.ts`

buildReplayRequest()

The app turns a replay action plus params into a request, reading secrets from the in-memory session.

faithful fingerprint

Headers and stable params learned by diffing the eight most recent real captures of that endpoint.

sendable headers

HTTP/2 pseudo-headers (`:method`, `:authority`) and non-byte values are dropped — undici rejects both.

fetch

20s abort covering the body read. Any HTTP status comes back as a capture; only a network failure throws.

stored as source='replay'

Redacted, ingested and parsed like live traffic, so a replay is visible in the same table.

MULTI-STEP FLOWS

`sluice replay --flow · sluice_replay_flow`

Learned offline

`learn-flows` clusters observed bursts into templates. MCP only reads and replays them, and templates never train on replay traffic.

Paced, not chained

Each step waits the median offset measured *from the primary*, capped at 2s; skipping a soft companion cannot desync the rest. Whole flow: 120s.

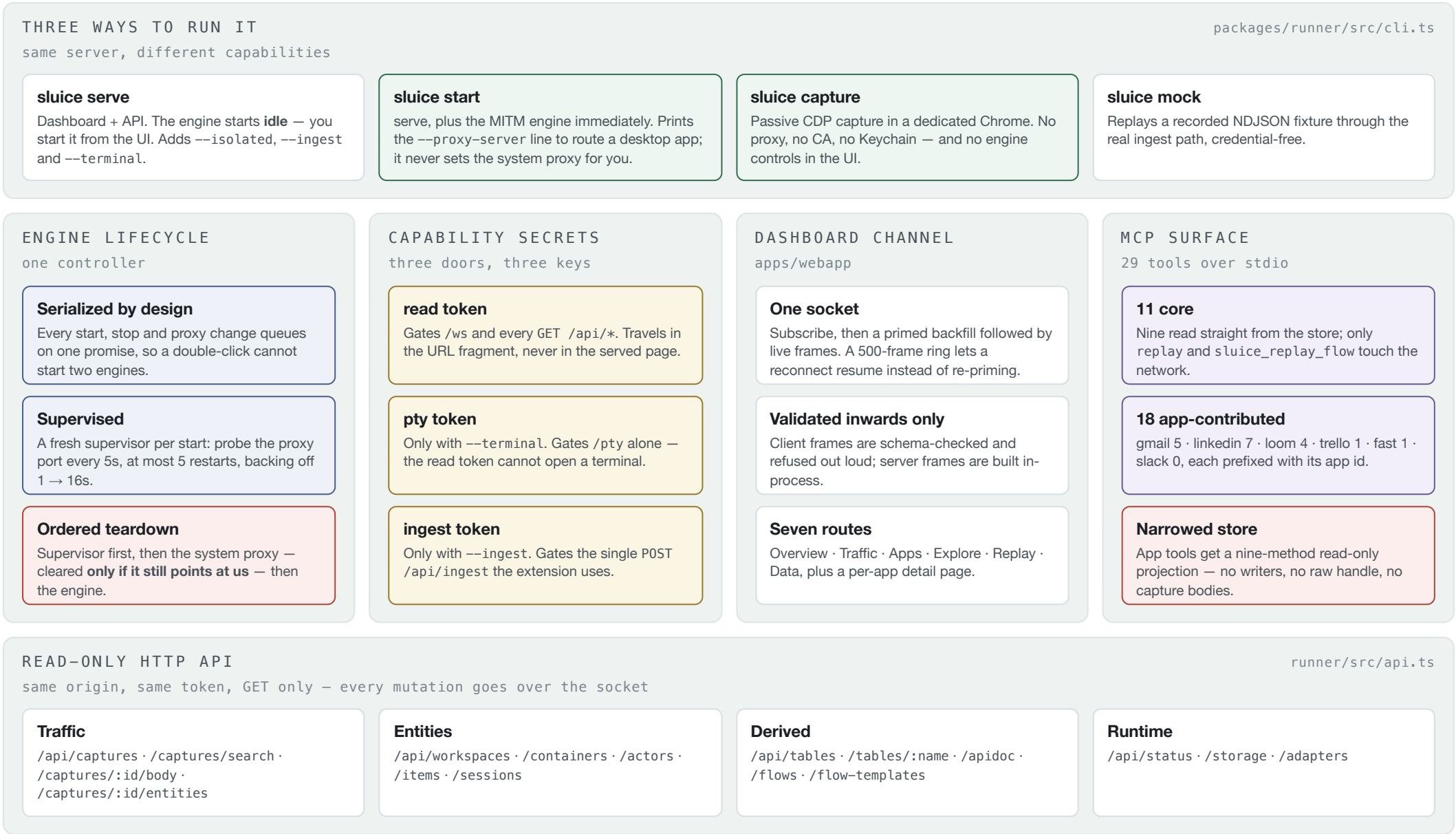
One extra rail

Each step is also checked against the owning adapter's host list, and an unresolvable path id fails the build rather than guessing.

Credential refresh sits **above** the rails, never inside them: on an auth failure the session is re-extracted once and the request rebuilt from the fresh one — both attempts recorded. A flow restarts from the top rather than resuming, because earlier steps may already have bound the stale session.

Control plane & consumers

What the runner exposes, to whom, and behind which secret



READ-ONLY HTTP API

same origin, same token, GET only — every mutation goes over the socket

Traffic

`/api/captures · /captures/search · /captures/:id/body · /captures/:id/entities`

Entities

`/api/workspaces · /containers · /actors · /items · /sessions`

Derived

`/api/tables · /tables/:name · /apidoc · /flows · /flow-templates`

Runtime

`/api/status · /storage · /adapters`

Every listener binds 127.0.0.1, checks the Host and Origin headers, compares its token in constant time, and serves pages under a CSP that permits no external origin. Table names reaching SQL are matched against the live schema first, so only materialized `<app>_*` tables are reachable.